GitHub Link :

https://github.com/AG0606/Intermediate-Code-Generator-Backpatching

Compiler Design Project | Final Review

Ashutosh Gupta – 24BCE2983 | Dhruv Mathur – 24BCE2504 | Somya Raj Singh Dalawat – 24BCE3001

Intermediate Code Generator for Control-Flow Statements

One-pass backpatching with symbol table, CFG construction and optimization

Compiler Design Project | Final Review

Ashutosh Gupta – 24BCE2983 | Dhruv Mathur – 24BCE2504 | Somya Raj Singh Dalawat – 24BCE3001

The Problem: Forward Jumps

Why targets are unknown

- A one-pass compiler emits code while parsing
- In if (B) S1 else S2, code for B comes before S1, S2 and the exit exist
- Symbolic labels need a second resolving pass and a label table
- Inherited label attributes are unavailable in bottom-up (LR) parsing

```
if (a < b && c < d) x = 1;

0: if a < b goto ?
1: goto ?
2: if c < d goto ?
3: goto ?
4: x = 1
5: ...
```

Backpatching: emit the jump with an empty target, remember its index in a list, fill it in once the target is known. Each jump is patched exactly once, so total work is O(n).

Objectives and Deliverables

Backpatching core

- truelist, falselist, nextlist; makelist, merge, backpatch; markers M and N

Full C-style control flow

- if, if-else, while, for, do-while, switch-case, break, continue

Semantic analysis

- Scoped symbol table, type checking, shadowing, error diagnostics

Analysis and optimization

- Basic blocks, CFG export (DOT and HTML), constant folding, jump chaining, dead-code removal

Delivered as a modular C project built with Flex and Bison, driven by a single CLI.

Survey 1/3: IR and Program Graphs

MLIR (Lattner et al.)

CGO 2021. Reusable, extensible compiler infrastructure for domain-specific compilers

Idea: Many IR levels in one framework, which cuts software fragmentation across compilers.
Link: Confirms the IR is the central contract between front end and optimizer, as our quadruple array is.
Gap: Targets large, multi-level, multi-target systems; it does not show how a small one-pass LR front end builds its IR.

ProGraML (Cummins et al.)

ICML 2021. Graph-based program representation for data-flow analysis

Idea: Programs as graphs capturing control, data and call relations; ML models struggle on basic data-flow tasks.
Link: Its control-flow edges mirror our basic-block CFG (true, false, jump, fallthrough).
Gap: Consumes an existing LLVM IR; it does not generate that IR from source or handle jump patching.

Survey 2/3: Syntax Error Recovery

98.37%

of 200,000 invalid Java files repaired within 0.5 s

435,812

error locations reported, vs 981,628 for panic mode

Don't Panic! (Diekmann and Tratt)

ECOOP 2020, pp. 6:1-6:32. CPCT+ error recovery for LR parsers

Idea: Panic-mode recovery causes cascades of spurious errors. CPCT+ reports the full set of minimum-cost repair sequences and picks one that causes fewer follow-on errors.
Link: Our parser is a Bison LR parser, so the same cascade problem applies to error_reporter.c and yyerror().
Gap: CPCT+ is a parser-level repair algorithm only. It does not couple recovery with semantic checks, scopes or code generation. Integrating it is future work for us.

Survey 3/3: Optimization and Correctness

DCE as a lens

Theodoridis, Rigger, Su. ASPLOS 2022

Idea: Insert markers in basic blocks, compute live/dead blocks, and judge compilers by how many dead blocks they remove.
Link: Our optimizer removes unreachable blocks found by CFG reachability.

DCE can delete live code

Xdead. ICSE-NIER 2024

Idea: Differential testing and symbolic execution found erroneous removal of live code, with two miscompilation bugs in LLVM.
Link: Motivates validating that our dead-code pass preserves program behavior.

LLM Compiler

Cummins et al. Meta AI, arXiv 2024

Idea: Models trained on 546B tokens of LLVM-IR and assembly reach 77% of an autotuner's code-size gains.
Link: Shows the IR-level optimization trend; quadruples are the same kind of artifact at small scale.

Survey Summary

Work	Year / Venue	Focus	Our module
MLIR	2021 / CGO	Multi-level IR infrastructure	quad.c, IR design
ProGraML	2021 / ICML	Graph program representation	cfg.c
Don't Panic! (CPCT+)	2020 / ECOOP	LR syntax error repair	error_reporter.c, parser.y
DCE lens	2022 / ASPLOS	Missed optimizations via dead blocks	optimizer.c
Xdead	2024 / ICSE-NIER	Live code wrongly deleted	optimizer.c, tests
LLM Compiler	2024 / arXiv	LLM-based IR optimization	Future work

Search note: recent peer-reviewed work on backpatching itself is scarce; the technique is mostly documented in textbooks and course material. The survey therefore covers the stages around it.

Research Gap and Our Contribution

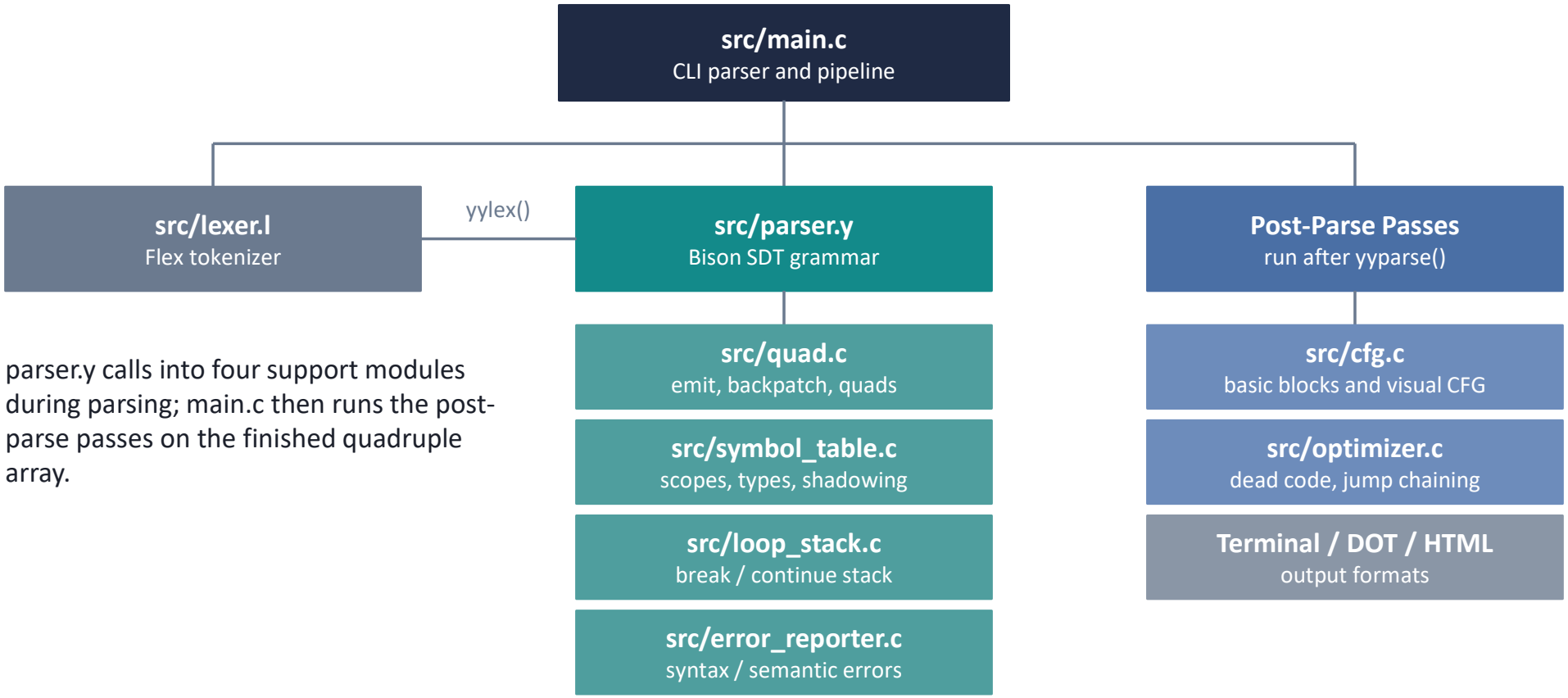
Gap in recent work

- Recent papers target LLVM-scale IRs, learned optimizers or isolated stages
- Parsing, scoping, IR generation, CFG and optimization are rarely shown together in one small, runnable pipeline
- Optimization correctness is an open concern (Xdead)

What this project adds

- One-pass backpatching generator for all C control-flow constructs
- Scoped symbol table, type checks and diagnostics feeding the same pipeline
- CFG export and an optimizer built on CFG reachability
- 13-program test harness to check behavior is preserved

System Architecture



parser.y calls into four support modules during parsing; main.c then runs the post-parse passes on the finished quadruple array.

End-to-End Execution

1. main() initializes symtab and loop stack
2. yyparse() pulls tokens from yylex()
3. Productions update symtab, loop stack and emit quads
4. Unoptimized 3AC and symbol table printed
5. cfg_build() partitions blocks, exports cfg.dot / cfg.html
6. optimize_quads() folds, chains, removes dead code
7. Optimized 3AC and metrics report

Invocation

- ./icg program.c --all
- ./icg < input.txt
- Flags: --cfg, -O, --symtab, --quads, --strict, --trace, --json
- Input from file or stdin

Backpatching Engine (quad.c)

```
emit(op,a1,a2,res)
  append quad, return index
makelist(i)    -> [i]
merge(p1,p2)   -> p1 + p2
backpatch(L,t)
  set target of every quad
  in L to t
newtemp()      -> t1, t2...
print_quads()
print_quad_table()
JSON export
```

Attributes

- B.truelist / B.falselist
- S.nextlist
- Markers: M records nextquad, N emits goto ?

Short-circuit rules

- B1 && M B2: patch B1.true to M
- B1 || M B2: patch B1.false to M
- !B swaps the lists; no boolean value is ever computed

Language and Grammar Coverage

Statements

- Assignment and declarations: int, float, char, bool
- if, if-else, nested conditionals
- while, for, do-while
- switch-case with break
- break and continue bound to innermost loop or switch
- Blocks with C scoping { }

Expressions

- Arithmetic with temporaries t1, t2, ...
- Relational: < <= > >= == !=
- Logical: &&, ||, ! with short-circuit
- Type checking on assignments and arithmetic
- Marker non-terminals keep the grammar LR-friendly

Symbol Table and Scoping

```
int x = 1;        // x
{
  int x = 2;      // x_s2
  y = x + 1;      // uses x_s2
}
z = x;            // uses x
```

Inner shadowed variables get unique IR names so quadruples always reference the right instance.

symbol_table.c

- Hash table with chained buckets
- Tracks type, line number, init state, scope
- symtab_enter_scope() on { and function entry
- symtab_exit_scope() on }
- symtab_insert() on declarations
- symtab_lookup() on every identifier use
- symtab_get_ir_name() when emitting quads

Break, Continue and Switch Context

loop_stack.c

- loop_stack_push(CTX_FOR | CTX_WHILE | ...) via mid-rule markers
- handle_break(): emit goto ?, add to top breaklist
- handle_continue(): emit goto ?, add to top continuelist
- loop_stack_pop() when the construct ends

Patching at loop exit

- continuelist backpatched to the loop update step
- breaklist merged into the exit nextlist
- Nested constructs resolve to the innermost context automatically
- break outside a loop is a semantic error

Worked Trace: Short-Circuit and If-Else

```
if (a < b && c < d) x = 1;

0: if a < b goto 2
1: goto 5
2: if c < d goto 4
3: goto 5
4: x = 1
5: end
```

```
if (a < b) x = 1; else x = 2;

0: if a < b goto 2
1: goto 4
2: x = 1
3: goto 5
4: x = 2
5: end
```

Left operand false jumps straight to the exit; falselist {1,3} is merged and patched once. In if-else, N emits quad 3 to skip the else branch, and its nextlist is patched to 5.

Worked Trace: Nested while + if-else

```
while (a < b)
  if (c < d) x = 1;
  else x = 2;

0: if a < b goto 2
1: goto 8
2: if c < d goto 4
3: goto 6
4: x = 1
5: goto 0
6: x = 2
7: goto 0
8: end
```

Patching across two levels

- Quad 5 (inner N) and the else branch fall through to the loop jump
- Quad 7 is the back-edge to the condition at 0
- Loop falselist {1} exits to 8
- Matches the algorithm in the Dragon Book, Chapter 6

Basic Blocks and Control Flow Graph

1

Find leaders

Quad 0, jump targets, instruction after a jump or return, function entries

2

Partition

Quads between leaders become BasicBlock structures

3

Add edges

True, False, Jump and Fallthrough

4

Reachability

BFS from block 0 and function entries marks dead blocks

cfg.c outputs

- cfg_print_blocks(): block connectivity table in the terminal
- cfg_export_dot("cfg.dot"): Graphviz; cfg_export_html("cfg.html"): interactive Mermaid.js viewer

Intermediate Code Optimization

Constant folding

- t1 = 10 + 20 becomes t1 = 30
- Applies to arithmetic on integer literals

Jump chaining

- goto L1 where L1 is goto L2 becomes goto L2
- Shortens control paths

Dead code removal

- Unreachable blocks from CFG reachability are removed
- Redundant goto i+1 is deleted
- Array compacted; all jump targets re-indexed

Diagnostics and Error Reporting

error_reporter.c

- report_syntax_error(): called from yyerror() with token and location
- report_semantic_error() and report_semantic_warning(): called from symbol_table.c, loop_stack.c and parser.y
- Centralized formatting with line numbers
- Global settings shared via common.h

Errors caught

- Use of an undeclared identifier
- Type mismatch in assignment or arithmetic
- break or continue outside a loop or switch
- Redeclaration within the same scope
- --strict mode for stricter checking

Build and Test Infrastructure

13

test programs run by run_tests.py against the compiled binary

Test coverage

- Conditionals and short-circuit cases
- Nested loops
- Switch statements
- C scoping and shadowing
- Deep nesting and mixed boolean conditions

Build

- Makefile: bison -d, flex, gcc -Wall -Isrc
- build.ps1 and build.bat for Windows
- README.md and PROJECT_REPORT.md

Conclusion and Future Work

Achieved

- Single-pass backpatching for all C control-flow constructs
- Scoped symbol table with shadowing and type checks
- Basic-block CFG with DOT and HTML export
- Constant folding, jump chaining, dead-code elimination
- Modular code base with automated tests

Future work

- Functions and arrays
- Further optimizations: copy propagation, common subexpressions
- Target code generation from the optimized quads
- Richer error recovery

References

- Lattner et al. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. CGO 2021.
- Diekmann, Tratt. Don't Panic! Better, Fewer, Syntax Errors for LR Parsers. ECOOP 2020.
- Cummins et al. ProGraML: A Graph-based Program Representation for Data Flow Analysis and Compiler Optimizations. ICML 2021.
- Theodoridis, Rigger, Su. Finding Missed Optimizations through the Lens of Dead Code Elimination. ASPLOS 2022.
- Beyond a Joke: Dead Code Elimination Can Delete Live Code. ICSE-NIER 2024. doi:10.1145/3639476.3639763
- Cummins et al. Meta Large Language Model Compiler: Foundation Models of Compiler Optimization. arXiv:2407.02524, 2024.
- Algorithm source: Aho, Lam, Sethi, Ullman. Compilers: Principles, Techniques, and Tools, 2nd ed., 2006, Ch. 6.